Understanding Python Decorators

Changing code structure without adding new lines

1. The Bridge: From Closures to Decorators

What is a Decorator?

• A decorator is a callable that takes another function as an argument, extends its behavior, and returns a modified function object without altering the original code.

• Closures make this possible: Because inner functions capture outer enclosing scopes, an inner 'wrapper' function can close over and execute the passed-in target function.

• Manual Reassignment: Before using syntactic sugar, decorators are manually assigned by passing the function as a parameter to the outer decorator.

```python
# A simple manual decorator structure
def null_decorator(func):
    def wrapper():
        print('Before execution')
        func()  # Executes closed-over target
        print('After execution')
    return wrapper

def greet():
    print('Hello World')

# Manual decoration / reassignment
greet = null_decorator(greet)
greet()  # Prints log borders dynamically!
```

2. Declarative Wrappers: The @ Syntax

Declarative Convenience

• The '@' symbol is purely syntactic sugar. Putting '@decorator' on top of a function definition tells Python to automatically execute the manual reassignment behind the scenes.

• Immediate Evaluation: Python evaluates the decorator immediately when the module is imported and the function is parsed, not when the function is eventually invoked.

• Enhanced Readability: It separates cross-cutting execution concerns (like security or logging) from core business logic, keeping functions clean and single-focused.

```python
# Declaring the decorator wrapper
def logged(func):
    def wrapper():
        print(f'Invoking: {func.__name__}')
        func()
    return wrapper

# Syntactic sugar equivalent to greet=logged(greet)
@logged
def greet():
    print('Greetings, delegates!')

greet()  # Automatically logs its invocation!
```

2a. Transforming Return Values

Changing the Output

- A wrapper can do more than print before and after a call: it can take the target's result and return a changed version.
- Here uppercase_decorator calls the original function, converts the returned string to upper case, and returns the new value.
- The manual form and the @ form behave identically: the name now refers to the wrapper, so every call returns upper-case text.

```
def uppercase_decorator(function):
    def wrapper():
        result = function()    # call target
        return result.upper()  # change output
    return wrapper

def say_hi():
    return 'hello there'

decorated = uppercase_decorator(say_hi)
print(decorated())   # HELLO THERE

@uppercase_decorator  # same, using @
def say_bye():
    return 'goodbye'

print(say_bye())     # GOODBYE
```

3. Generic Wrapping with *args and **kwargs

Signature Agnosticism

• Basic wrappers fail when the decorated target accepts arguments. If our target expects 'a' and 'b', but the wrapper closure expects no arguments, Python raises a TypeError.

• The Solution: To construct a generic decorator capable of wrapping any function, the inner wrapper closure must accept '*args' and '**kwargs'.

• Parameter Forwarding: Arguments are passed directly from the wrapper execution boundary to the closed-over target function via 'func(*args, **kwargs)'.

```python
def audit_log(func):
    # wrapper accepts ANY arguments
    def wrapper(*args, **kwargs):
        print(f'Target {func.__name__} started.')
        result = func(*args, **kwargs) # Forward
        print(f'Target completed.')
        return result  # Preserve return values!
    return wrapper

@audit_log
def add_numbers(a, b):
    return a + b

print(add_numbers(10, 20))  # Correctly prints 30
```

4. Preserving Metadata: wraps()

The Metadata Pitfall

• Because wrapping replaces the target function reference with the inner wrapper closure, original metadata—docstrings, names, and typing annotations—is lost.

• If you inspect 'func.__name__' after decoration, it returns 'wrapper' rather than the original name. This breaks debugging tools, auto-doc systems, and introspections.

• The Solution: Use Python's built-in '@functools.wraps' decorator on the inner wrapper closure to automatically copy original attribute data back.

```python
from functools import wraps

def audited(func):
    @wraps(func)  # Restores original attributes
    def wrapper(*args, **kwargs):
        '''Audits execution'''
        return func(*args, **kwargs)
    return wrapper

@audited
def calculate_total():
    '''Computes shopping basket items'''
    pass

print(calculate_total.__name__) # calculate_total
print(calculate_total.__doc__)  # Computes...items
```

5. Stacking Decorators: Execution Order

Bottom-Up Nesting

• You can stack multiple decorators on a single function. They execute in a nested, bottom-up order, which maps directly to functional nesting.

• Execution Trace: Placing '@bold' above '@italic' over 'greet()' resolves as 'greet = bold(italic(greet))'.

• Core Wrapper Order: The top decorator wraps the output of the decorator below it. When greet() runs, it hits bold first, which calls italic, which finally calls the greet target.

```
def bold(func):
    def wrapper():
        return f'<b>{func()}</b>'
    return wrapper

def italic(func):
    def wrapper():
        return f'<i>{func()}</i>'
    return wrapper

@bold
@italic
def greet():
    return 'Hello'

print(greet()) # <b><i>Hello</i></b>
```

5a. When Stacking Order Matters

Order Changes the Result

- The decorator nearest the def is applied first; its output becomes the input of the decorator above it.
- Here the text is converted to upper case first, and the upper-case string is then split into a list of words.
- Reversing the order fails: split_string returns a list, and a list has no upper() method, so Python raises an AttributeError.
- Rule: place each decorator where the value it receives has the type it expects.

```
def uppercase_decorator(function):
    def wrapper():
        return function().upper()
    return wrapper

def split_string(function):
    def wrapper():
        return function().split()
    return wrapper

@split_string          # applied second
@uppercase_decorator   # applied first
def say_hi():
    return 'hello there'

print(say_hi())  # ['HELLO', 'THERE']

# Swapping the two @ lines raises:
# AttributeError: 'list' object
# has no attribute 'upper'
```

6. Advanced: Decorators with Arguments

Decorator Factories

• To configure your decorator at the declaration boundary—e.g., '@repeat(num_times=3)'—you must add another nested outer function.

• The Three-Tier Architecture:
1. Factory: Receives configuration arguments.
2. Decorator: Receives the target function.
3. Wrapper: Receives invocation parameters.

• Double Invocation: '@repeat(3)' executes the factory, returning a custom decorator that Python then applies to the target.

```python
def repeat(num_times):  # Tier 1: Factory
    def decorator_repeat(func): # Tier 2: Decorator
        @wraps(func)
        def wrapper(*args, **kwargs): # Tier 3: Wrapper
            for _ in range(num_times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator_repeat

@repeat(num_times=3) # factory evaluated first
def hello(name):
    print(f'Hello {name}')

hello('Bob')  # Prints 'Hello Bob' 3 times!
```

7. Structural Comparison: Closures vs. Decorators

Dimension	Closures	Decorators
Primary Purpose	Capturing stateful environments inside lightweight function namespaces.	Extending, intercepting, or caching behaviors of other functions cleanly.
Syntax & Style	Explicit nesting with manual return of inner scopes.	Declarative sugar using the '@' macro placed on top of targets.
Invocation Scope	Invoked dynamically by passing variables to factory creators.	Parsed immediately at import time; wrapper triggered at invocation.
Sweet-spot Use Case	Stateful counters, math factories, and lightweight functional classes.	Logging cross-cutting concerns, timing, access auth, and caching.

8. Decorators in Real Projects

Where You Will Meet Them

- Logging and auditing: record each call to a sensitive function.
- Timing and profiling: measure how long a function takes to run.
- Access control: Flask-Login's @login_required checks the user before a page runs.
- Caching: functools.lru_cache stores the results of expensive calls.
- Web routing and testing: Flask's @app.route and pytest's @pytest.fixture.
- Built-in: @property, @staticmethod, @classmethod and @dataclass.

```python
from functools import lru_cache

@lru_cache(maxsize=None)  # store results
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(80))   # 23416728348467685
print(fib.cache_info().hits)   # 78

# Without the cache, fib(80) would
# need more than 10**16 calls.
```